Informe de Diseño y Análisis: Torneo de Komikan Concurrente

Tarea 2 – Sistemas Operativos | **Integrantes:** Nicolas Cuevas y Kristofer Muñoz
Repositorio GitHub: https://github.com/sleephv/T2_SO_Nicolas_Cuevas_Kristofer_Munoz

1. Diseño del Esquema de Sincronización y Recursos Compartidos

El sistema implementa una arquitectura concurrente multicapa en C++17 mediante primitivas POSIX (`pthread_mutex_t`, `pthread_cond_t` y `sem_t`) para coordinar el emparejamiento de N clientes concurrentes en un máximo de M mesas activas y arbitrar la competencia en tiempo real de 13 fichas autónomas por partida (1 Puma y 12 Perros).

1.1. Modelado del Tablero y Patrón Monitor por Mesa

El tablero tradicional de Komikan se representó como un grafo estático no dirigido de exactamente **28 intersecciones**: el nodo `0` corresponde a la cúspide del Trono (`T`), los nodos `1` y `2` a las posiciones intermedias del triángulo superior, y los nodos `3` al `27` a la cuadrícula principal de 5×5. Para proteger el estado compartido de cada partida sin incurrir en bloqueos cruzados, cada instancia de `TableroKomikan` se diseñó bajo el **Patrón Monitor**:

- **Exclusión Mutua Atómica (`mutex_tablero`):** Cada vez que el hilo del Puma o uno de los 12 hilos de los Perros finaliza su ciclo de decisión (`usleep` aleatorio entre `min_delay` y `max_delay`, por defecto 50–300 ms), adquiere `mutex_tablero`. Dentro de esta sección crítica de duración sub-milisegundo, el hilo verifica si la partida continúa activa, consulta las casillas adyacentes vacías (o saltos válidos de captura sobre un perro en el caso del Puma) y reclama atómicamente la casilla destino actualizando el arreglo `casillas[28]` y su posición. Si dos fichas despiertan en el mismo milisegundo y eligen el mismo nodo libre, el mutex serializa el acceso garantizando que la primera reclame el slot y la segunda observe inmediatamente el estado actualizado sin condiciones de carrera (*Race Conditions*).
- **Supervisión Pasiva del Juez Árbitro (`cond_juez`):** Cada mesa cuenta con un hilo Juez independiente que evita por completo el *busy-waiting*. El Juez permanece bloqueado en `pthread_cond_wait(&cond_juez, &mutex_tablero)` y solo es despertado mediante `pthread_cond_signal` cuando una ficha concreta un movimiento válido, ocurre una desconexión o se recibe una señal de cierre. Al despertar, imprime el estado del tablero en consola y en `torneo.log`, y evalúa las condiciones de término (captura de ≥ 6 perros o acorralamiento total del Puma).

1.2. Sincronización del Servidor Matchmaker y Límite de M Mesas

La coordinación global entre los N clientes y las M mesas se divide en tres primitivas desacopladas:

Primitiva POSIX	Ámbito	Función en la Arquitectura de Sincronización
`sem_mesas_disponibles`	Servidor (M)	Semáforo contador inicializado en M. Garantiza espera pasiva (`sem_wait`) cuando las M mesas están ocupadas y libera cupos (`sem_post`) al concluir cada partida.
`mutex_mm` + `cond_lobby`	Matchmaker	Protege la cola FIFO de llegada (`cola_espera`) y el arreglo `mesa_libre`. Suspende pasivamente a clientes sin pareja y retira limpiamente al jugador sobrante si N es impar.
`mutex_tablero` + `cond_juez`	Por Mesa	Sincroniza los movimientos simultáneos de las 13 fichas y despierta al Juez tras cada jugada o evento de fallo.
`mutex_consola`	Salida / Log	Serializa la escritura del tablero en `stdout` y `torneo.log` para evitar entrelazamiento de líneas entre mesas concurrentes.

2. Análisis de Riesgos de Concurrencia (Deadlock, Livelock y Starvation)

2.1. Prevención de Interbloqueos (*Deadlock*)

En un diseño alternativo basado en 28 mutexes individuales (uno por casilla), si el Perro A intenta moverse de la casilla $u \to v$ reteniendo `mutex[u]` mientras el Puma o el Perro B intenta moverse de $v \to u$ reteniendo `mutex[v]`, se produciría un *Deadlock* clásico por espera circular (o requeriría ordenar estrictamente la adquisición por índice menor a mayor e involucrar hasta 3 mutexes simultáneos en un salto de captura del Puma). Nuestro diseño de **Monitor por Mesa** erradica estructuralmente el *Deadlock* al romper dos condiciones necesarias de Coffman:

1. **Eliminación de *Hold and Wait* en el tablero:** Ninguna ficha retiene un cerrojo de casilla mientras solicita otro; la validación de adyacencia y el reclamo del slot ocurren en un único paso atómico bajo `mutex_tablero`.

2. **Jerarquía estricta de cerrojos (*Lock Ordering*):** Cuando el Juez debe imprimir en consola o escribir en el log mientras posee `mutex_tablero`, adquiere `mutex_consola` como cerrojo hoja y lo libera inmediatamente. Ningún hilo del sistema adquiere jamás `mutex_tablero` estando dentro de `mutex_consola`, imposibilitando ciclos en el grafo de asignación de recursos.

2.2. Prevención de *Livelock* y *Starvation* (Inanición)

- **Mitigación de *Starvation*:** Dado que existen 12 hilos de Perros frente a 1 solo hilo del Puma, podría temerse que el Puma sufra inanición al competir por el mutex de la mesa. Sin embargo, el tiempo de retención de `mutex_tablero` es del orden de ≈ 1–5 μs (solo operaciones en memoria RAM), mientras que el ciclo de pensamiento fuera de la sección crítica es de 50 000 a 300 000 μs (`usleep`). Así, la fracción de tiempo que un hilo ocupa el cerrojo es $< 0{,}01\,\%$, permitiendo que el planificador *Completely Fair Scheduler* (CFS) de Linux y la cola FIFO interna de `pthread_mutex_lock` otorguen acceso inmediato al Puma sin inanición.
- **Mitigación de *Livelock*:** Para evitar que dos fichas oscilen sincrónicamente bloqueándose mutuamente el paso, cada hilo emplea la función reentrante `rand_r(&seed_local)` con una semilla derivada de su identificador y del ID de mesa. Esta dispersión estocástica en los tiempos de retardo y en la elección de vecinos libres rompe cualquier simetría temporal. Además, el Puma prioriza determinísticamente los saltos de captura sobre los movimientos simples, acelerando la convergencia de las partidas.

3. Resiliencia y Evaluación del Diseño

3.1. Resiliencia ante Fallos de Jugadores y Señal `SIGINT` (`Ctrl+C`)

- **Recuperación ante Desconexión/Fallo (Sección 4.1):** Los hilos `hilo_puma` e `hilo_perro` registran un manejador de limpieza POSIX mediante `pthread_cleanup_push(liberar_mutex_en_fallo, &ctx)`. Si un hilo aborta o falla internamente (`pthread_exit`) mientras evalúa un movimiento, el manejador desbloquea automáticamente `mutex_tablero`, marca `anomalia_detectada = true` y emite `pthread_cond_signal(&cond_juez)`. El Juez detecta la anomalía de inmediato, clausura la partida declarando al oponente ganador por abandono y ejecuta `pthread_join` sobre los 13 hilos antes de destruir limpiamente las primitivas de la mesa.
- **Clausura Ordenada ante `Ctrl+C` (Sección 4.2):** Al capturar `SIGINT` con `sigaction`, se activa la bandera atómica `sigint_recibido = 1`. El hilo vigía invoca `notificar_cierre_emergencia()`, vaciando la sala de espera, señalizando `cond_lobby` y despertando a los Jueces de todas las mesas en curso. El proceso principal recolecta la totalidad de los hilos mediante `pthread_join`, libera la memoria dinámica (`delete`) y destruye semáforos y mutexes garantizando cero fugas de memoria y cero recursos huérfanos.

3.2. Balance entre Complejidad, Rendimiento (*Throughput*) y Concurrencia

- **Nivel de Concurrencia Inter-Mesas e Intra-Mesa:** Las M mesas activas operan con paralelismo real completo entre sí, ya que cada `TableroKomikan` posee su propio mutex independiente. Dentro de cada mesa, los 13 hilos ejecutan sus retardos de decisión (50–300 ms) de forma 100 % concurrente en paralelo y solo serializan el instante microscópico de actualización de la matriz.
- **Evaluación Frente a Alternativas de Grano Fino:** Implementar 28 mutexes por tablero incrementaría drásticamente la complejidad algorítmica (requiriendo bloqueo simultáneo ordenado de 2 casillas para movimientos simples y 3 casillas para capturas, además de un cerrojo adicional para que el Juez lea un estado consistente del tablero sin *torn reads*). En cambio, el Monitor por Mesa ofrece una complejidad de mantenimiento baja, consistencia absoluta para las instantáneas del Juez Árbitro y una latencia de adquisición prácticamente nula, logrando un balance óptimo para el Torneo de Komikan Concurrente.